Task-3 Report

Model Validation, Cross-Validation and Hyperparameter Tuning for House Price Prediction

Intern Name: Krishna

Project: House Price Prediction using Machine Learning

Dataset: California Housing Dataset

Tools Used: Python, Jupyter Notebook, Pandas, NumPy, Scikit-learn, Matplotlib

1. Introduction

Machine Learning models should not only perform well on the training data but should also make accurate predictions on new and unseen data. A model that performs well only on training data is not reliable for real-world applications. Therefore, validating the model properly is one of the most important steps in the Machine Learning workflow.

In Task-2, a House Price Prediction model was developed using different regression algorithms. However, evaluating a model using only a single train-test split may not provide reliable information about its actual performance.

The objective of Task-3 is to improve the reliability of the model by introducing professional Machine Learning techniques such as overfitting detection, cross-validation, hyperparameter tuning, and scientific model selection. These techniques help in building models that generalize better to unseen data and provide more trustworthy predictions.

The California Housing Dataset was used throughout this task to maintain consistency with the previous task. The dataset contains several housing-related features such as median income, average number of rooms, housing age, population, latitude, and longitude. The target variable is the median house value.

2. Dataset Preparation

The California Housing Dataset was loaded using Scikit-learn's built-in dataset module. After loading the dataset, the feature columns and target column were separated.

The input features include:

- Median Income
- House Age
- Average Rooms

- Average Bedrooms
- Population
- Average Occupancy
- Latitude
- Longitude

The target variable is:

- Median House Value (HousePrice)

Before training the models, feature scaling was performed using the StandardScaler. Feature scaling standardizes all numerical features so that every feature contributes equally during model training. Since different features have different ranges, scaling improves learning stability and optimization performance, especially for regression algorithms.

The dataset was then divided into training and testing sets using an 80:20 ratio. The training data was used to train the models, while the testing data was reserved for evaluating model performance on unseen examples.

3. Overfitting Analysis

One of the major objectives of this task was to detect overfitting.

Overfitting occurs when a machine learning model memorizes the training dataset instead of learning the actual relationship between features and the target variable. As a result, the model achieves excellent performance on training data but performs poorly on unseen testing data.

To analyze overfitting, a Decision Tree Regressor was trained using the training dataset.

The model's performance was then evaluated separately on:

- Training Data
- Testing Data

The Root Mean Squared Error (RMSE) was calculated for both datasets.

During the analysis, it was observed that the training RMSE was significantly lower than the testing RMSE. This indicates that the Decision Tree learned many patterns specific to the training dataset, including noise, which reduced its ability to generalize to new data.

This behaviour clearly demonstrates overfitting.

In practical Machine Learning applications, an overfitted model is considered unreliable because it performs well only on previously seen data and fails to maintain similar performance on unseen datasets.

Therefore, reducing overfitting becomes an important objective before deploying any Machine Learning model.

4. Cross-Validation

Instead of relying on only one train-test split, cross-validation provides a much more reliable estimate of model performance.

In this project, 5-Fold Cross-Validation was performed using the `cross_val_score()` function.

The complete dataset was divided into five equal parts.

The training process was repeated five different times.

During each iteration:

- Four parts were used for training.
- One part was used for validation.

Every data sample became part of the validation set exactly once.

The RMSE obtained from all five folds was then averaged to obtain the final cross-validation score.

Cross-validation offers several advantages:

- It reduces dependency on one random train-test split.
- It provides a more stable estimate of model performance.
- It reduces evaluation bias.
- It improves confidence in model reliability.
- It helps compare different models fairly.

Since every observation participates in both training and validation, cross-validation provides a more scientific evaluation of model performance compared to a single train-test split.

5. Hyperparameter Tuning

After identifying overfitting, the next objective was to improve model performance through hyperparameter tuning.

Hyperparameters are configuration settings chosen before training begins. They directly influence how a Machine Learning algorithm learns from the data.

Unlike model parameters, hyperparameters are not learned automatically during training.

For the Decision Tree Regressor, the following hyperparameters were optimized:

Maximum Depth (max_depth)

This parameter controls the maximum depth of the tree.

A very deep tree may memorize the training dataset, causing overfitting.

Restricting the depth forces the model to learn more generalized decision rules.

Minimum Samples Split (min_samples_split)

This parameter specifies the minimum number of samples required before creating another split.

Larger values prevent unnecessary splitting and reduce model complexity.

Minimum Samples Leaf (min_samples_leaf)

This parameter determines the minimum number of samples that each leaf node must contain.

Increasing this value creates smoother decision boundaries and helps reduce overfitting.

To identify the best combination of these hyperparameters, GridSearchCV was used.

GridSearchCV systematically evaluates every possible combination of hyperparameter values.

Each combination is evaluated using 5-Fold Cross-Validation.

The model with the lowest average RMSE is automatically selected as the best model.

This approach eliminates manual trial-and-error and ensures that the selected model has been optimized scientifically.

6. Model Evaluation

After obtaining the best hyperparameters, the optimized Decision Tree model was trained again.

The optimized model was then evaluated using the testing dataset.

The following evaluation metrics were used:

Root Mean Squared Error (RMSE)

RMSE measures the average prediction error.

A lower RMSE indicates that the predicted house prices are closer to the actual values.

Lower RMSE always represents better model performance.

R^2 Score

The R^2 Score measures how much variation in house prices is explained by the model.

The score ranges from 0 to 1.

Values closer to 1 indicate better prediction capability.

A higher R^2 Score means the model explains a larger percentage of the variation in house prices.

The optimized Decision Tree produced improved testing performance compared to the untuned model, demonstrating that hyperparameter tuning successfully reduced overfitting.

7. Model Comparison

Three regression models were compared in this project:

- Linear Regression
- Ridge Regression
- Tuned Decision Tree Regressor

Linear Regression is simple, fast, and easy to interpret but assumes a linear relationship between features and the target variable.

Ridge Regression improves Linear Regression by introducing regularization, which helps reduce overfitting while maintaining model simplicity.

The Decision Tree Regressor can capture complex nonlinear relationships but is naturally prone to overfitting if not properly controlled.

After hyperparameter tuning, the Decision Tree achieved a much better balance between model complexity and prediction accuracy.

The comparison table included RMSE and R^2 values for all three models, allowing objective comparison based on quantitative performance metrics.

8. Final Model Selection Justification

The Tuned Decision Tree Regressor was selected as the final model.

Several reasons support this decision.

First, GridSearchCV automatically selected the best hyperparameter combination after evaluating multiple configurations using cross-validation.

Second, overfitting was significantly reduced by limiting the depth of the tree and controlling the minimum number of samples required for splitting and leaf creation.

Third, cross-validation confirmed that the optimized model maintained stable performance across multiple validation folds, increasing confidence in its ability to generalize to unseen data.

Finally, the tuned Decision Tree provided an excellent balance between prediction accuracy and model complexity.

Although Linear Regression and Ridge Regression are computationally simpler, they may not fully capture the nonlinear relationships present in housing data.

The tuned Decision Tree demonstrated superior flexibility while maintaining good generalization performance, making it the most suitable model for this project.

9. Conclusion

Task-3 introduced several professional Machine Learning practices that significantly improved the quality and reliability of the House Price Prediction system.

The project successfully demonstrated how to detect overfitting, perform cross-validation, optimize hyperparameters using GridSearchCV, evaluate models using RMSE and R^2 Score, and scientifically justify the selection of the final model.

These techniques are widely used in real-world Machine Learning projects because they improve model robustness and reduce the risk of poor performance on unseen data.

Overall, the optimized Decision Tree Regressor was selected as the final model because it provided better generalization, reduced overfitting, and achieved reliable predictive performance.

The knowledge gained during this task strengthens the understanding of professional model validation techniques and prepares the model for practical real-world deployment.